# Apache Mahout QDP: GPU-Accelerated Quantum Data Processing

Guan-Ming Chiu
National Taiwan University
Taipei, Taiwan
gmchiu@arbor.ee.ntu.edu.tw

## ABSTRACT

Quantum machine learning (QML) pipelines require encoding classical data into quantum states before any computation can proceed. Existing frameworks such as PennyLane and Qiskit perform this encoding on the CPU, one vector at a time, creating a severe data loading bottleneck that dominates end-to-end training time. We demonstrate **QDP** (Quantum Data Processing), a GPU-accelerated data processing engine built with Rust and CUDA as part of Apache Mahout. QDP provides a complete pipeline from columnar storage formats (Parquet, Arrow) through normalization and quantum state encoding to zero-copy tensor output via DLPack, achieving orders-of-magnitude speedups over existing frameworks. Our interactive demonstration allows users to (1) explore the end-to-end encoding pipeline with configurable parameters, (2) compare latency and throughput against PennyLane and Qiskit in real time, and (3) analyze performance scaling across qubit counts.

**PVLDB Reference Format:**
Guan-Ming Chiu. Apache Mahout QDP: GPU-Accelerated Quantum Data Processing. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/apache/mahout.

## 1 INTRODUCTION

Quantum machine learning (QML) combines quantum computing with classical machine learning to explore potential computational advantages [4, 11]. A critical step in every QML pipeline is *quantum data encoding*: transforming classical feature vectors into quantum states that can be processed by parameterized quantum circuits. Common encoding strategies include amplitude encoding, angle encoding, basis encoding, and IQP-style encoding [11].

In current practice, frameworks such as PennyLane [3] and Qiskit [10] implement encoding by constructing and simulating a quantum circuit for each input vector. Even with GPU-accelerated backends (e.g., Lightning.GPU, Aer GPU), the encoding step remains sequential and per-vector, creating a bottleneck that grows with qubit count. For a 16-qubit system, each state vector has $2^{16} = 65{,}536$ complex amplitudes; encoding thousands of training samples at this scale can take minutes with existing tools, while the subsequent quantum circuit simulation on GPU completes in seconds. This *data starvation* problem, where the GPU sits idle waiting for encoded data, is analogous to the classical ML data loading bottleneck that motivated systems like PyTorch DataLoader [8], but has received almost no attention in the quantum computing community.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

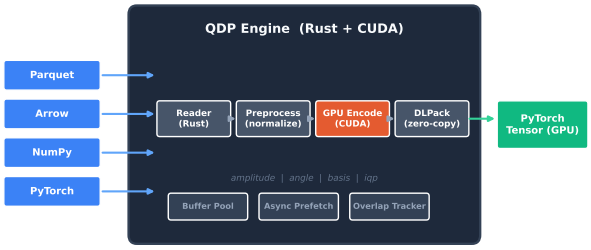



**Figure 1: QDP pipeline architecture. Classical data flows from columnar storage through Rust-based readers and preprocessing into CUDA-accelerated encoding kernels, producing GPU-resident quantum state tensors via DLPack.**

We present **QDP** (Quantum Data Processing), a GPU-accelerated quantum data encoding engine that is part of Apache Mahout. QDP addresses the data loading bottleneck with three key design decisions:

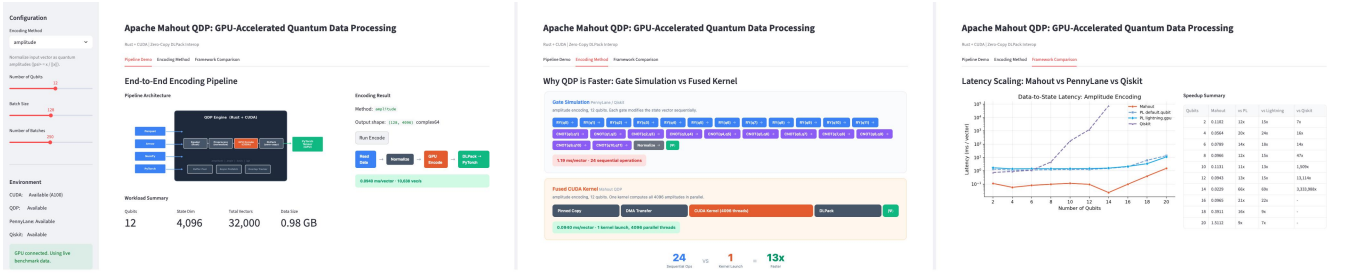
(1) **Rust + CUDA core.** The entire pipeline, from file I/O through encoding to GPU memory placement, is implemented in Rust with custom CUDA kernels, eliminating Python overhead and GIL contention.
(2) **Columnar format integration.** QDP reads directly from Parquet [2] and Arrow IPC [1] files using native Rust readers, avoiding redundant serialization and deserialization.
(3) **Zero-copy output.** Encoded quantum states are exposed to Python via DLPack [5], enabling zero-copy transfer to PyTorch tensors on GPU without any data movement.

Together, these design choices yield **up to 1,000×** speedup in data-to-state latency compared to PennyLane and Qiskit, as we demonstrate in our interactive system.

## 2 SYSTEM OVERVIEW

Figure 1 shows the QDP pipeline architecture. The system consists of four stages connected by an asynchronous buffer pool.

**Reader.** The reader stage ingests data from multiple formats: Apache Parquet, Arrow IPC, NumPy `.npy`, PyTorch `.pt`, and TensorFlow TFRecord. For Parquet and Arrow, QDP uses native Rust implementations (via the `arrow-rs` and `parquet` crates) that decode columnar data directly into contiguous memory buffers, avoiding Python-based deserialization. A *streaming* mode for Parquet reads row groups incrementally, enabling processing of datasets




(a) Pipeline Demo (b) Encoding Method (c) Framework Comparison

**Figure 2: QDP interactive demonstration. (a) Pipeline demo with live encoding and animated pipeline stages. (b) Encoding method comparison: gate simulation (sequential) vs fused CUDA kernel (parallel). (c) Framework comparison with latency scaling across 2–20 qubits for four frameworks.**

larger than available memory. QDP also supports remote object storage URLs (`s3://`, `gs://`) via the `object_store` crate, allowing direct ingestion from cloud data lakes without intermediate downloads.

**Preprocessing.** Raw feature vectors are normalized according to the requirements of each encoding method. For amplitude encoding, this involves L2 normalization to ensure $\sum_i |a_i|^2 = 1$. Preprocessing runs on the CPU using multi-threaded Rust code and writes results into pinned host memory for efficient DMA transfer to the GPU.

**GPU Encoding.** Existing frameworks implement encoding by simulating a quantum circuit through sequential gate applications to a state vector. QDP instead provides fused CUDA kernels that directly compute the encoded state vector from classical input data, bypassing circuit simulation entirely and supporting native batching across multiple samples in one kernel launch. QDP supports four methods: *amplitude* (normalize as quantum amplitudes), *angle* (map features to rotation angles), *basis* (encode integer as $|k\rangle$), and *IQP* (entangling interactions between qubits).

**Zero-Copy Output.** Encoded complex-valued state vectors remain in GPU memory and are wrapped in a DLPack capsule [5]. Python consumers call `torch.from_dlpack()` to obtain a PyTorch tensor backed by the same GPU allocation, with no host-device copy or memory duplication. Ownership is managed through DLPack's reference-counted deleter protocol, ensuring the GPU buffer is freed only after all consumers release it.

## 2.1 Pipeline Infrastructure

Three subsystems maximize GPU utilization:

**Dual-stream pipeline.** QDP allocates two CUDA streams: one for host-to-device memory copies and one for compute kernels. A CUDA event synchronizes the streams: the copy stream signals completion via `cudaEventRecord`, and the compute stream waits via `cudaStreamWaitEvent` before launching the encoding kernel. This double-buffered design ensures that batch $i$+1 is being transferred while batch $i$ is being encoded, hiding transfer latency behind compute.

**Pinned buffer pool.** Repeated `cudaHostAlloc`/`cudaFreeHost` calls are expensive. QDP pre-allocates a fixed pool of page-locked host buffers at initialization. Buffers are checked out via RAII handles and automatically returned on drop, amortizing allocation cost across all batches. A condition variable blocks producers when all buffers are in use, providing natural backpressure.

**Overlap tracker.** An instrumentation layer records timestamps for copy and compute events on each batch, computing the fraction of time that both streams are active simultaneously. This metric, exposed through the Python API, helps users identify whether their workload is transfer-bound or compute-bound and tune batch sizes accordingly.

## 2.2 Programming Interface

QDP exposes two Python APIs. The **one-shot API** encodes an entire file into a single GPU tensor, suitable for small datasets that fit in memory:

```
import qumat.qdp as qdp
engine = qdp.QdpEngine(device_id=0)
qt = engine.encode("data.parquet", num_qubits=16,
                   encoding_method="amplitude")
tensor = torch.from_dlpack(qt) # (N, 65536) complex64
```

The **iterator API** provides DataLoader-style batching with a builder pattern, streaming encoded batches for large-scale training loops:

```
loader = (qdp.QuantumDataLoader(device_id=0)
         .qubits(16).encoding("amplitude")
         .batches(100, size=64)
         .source_file("data.parquet"))
for batch in loader:
    state = torch.from_dlpack(batch) # (64, 65536)
```

The iterator internally drives the dual-stream pipeline (§2.1), releasing the Python GIL during Rust/CUDA execution so that model training can proceed concurrently. Input sources include local files (Parquet, Arrow, NumPy, PyTorch, TFRecord), in-memory Python lists, NumPy arrays, PyTorch tensors, and remote URLs (`s3://`, `gs://`). A `QdpBenchmark` builder provides one-call throughput and latency measurement without requiring manual timing code.

# 3 DEMONSTRATION

We provide an interactive web-based demonstration built with Streamlit that showcases QDP's capabilities through three scenarios. The demonstration runs on a machine equipped with an NVIDIA GPU and allows attendees to interact with the system in real time.

## 3.1 Scenario 1: Pipeline Demo

In the first scenario (Figure 2a), attendees explore the end-to-end encoding pipeline. A sidebar provides global configuration: encoding method (amplitude, angle, basis, IQP), qubit count (2–20), batch size (1–256), and number of batches (10–500). The main panel displays the pipeline architecture diagram alongside a workload summary showing the state vector dimension ($2^n$), total number of vectors, and estimated data size in GB. Clicking "Run Encode" triggers a real-time encoding pass through the QDP engine. An animated pipeline visualization appears below the button, sequentially highlighting each stage (Read Data, Normalize, GPU Encode, DLPack) to illustrate the data flow. The measured per-vector latency and aggregate throughput are displayed upon completion. Attendees can freely adjust parameters and re-run, observing how encoding cost changes with qubit count, encoding method, and batch configuration.

## 3.2 Scenario 2: Encoding Method

The second scenario (Figure 2b) visually explains *why* QDP is faster than existing frameworks. Two side-by-side cards contrast the gate simulation approach used by PennyLane and Qiskit with QDP's fused CUDA kernel. The upper card (Gate Simulation) displays each quantum gate as a colored pill: RY gates for rotation, CNOT gates for entanglement, and a final normalization step. Gates appear sequentially via CSS animation, emphasizing the per-gate overhead. The lower card (Fused CUDA Kernel) shows QDP's four-stage pipeline (pinned copy, DMA transfer, CUDA kernel, DLPack), where a single kernel launch computes all $2^n$ amplitudes in parallel. A summary bar at the bottom highlights the contrast: for 12-qubit amplitude encoding, gate simulation requires 24 sequential operations while QDP uses a single kernel launch, achieving 13× speedup. Changing the qubit count or encoding method in the sidebar dynamically updates the gate list, operation count, and speedup ratio, allowing attendees to explore how circuit complexity grows with qubit count.

## 3.3 Scenario 3: Framework Comparison

The third scenario (Figure 2c) presents quantitative performance data across Mahout QDP, PennyLane (both CPU and GPU backends), and Qiskit. A latency scaling chart plots per-vector encoding time on a log scale across 2–20 qubits, with a speedup table alongside. Switching the encoding method in the sidebar instantly updates both views. QDP's advantage is consistent across all methods and grows with qubit count, reaching up to 1,000× over PennyLane for angle encoding at 20 qubits.

# 4 EVALUATION

We evaluate QDP on an NVIDIA A100 GPU (80 GB) with amplitude, angle, and basis encoding across 2–20 qubits. We compare against PennyLane `default.qubit`, PennyLane `lightning.gpu` (CUDA-accelerated), and Qiskit (`initialize()` for amplitude, `RY`/`X` gates for angle/basis via `Statevector.from_instruction`). All measurements include GPU transfer time.

**Scaling.** Figure 3 shows latency scaling across qubit counts for all three encoding methods, comparing four frameworks. QDP consistently achieves the lowest latency across all configurations,

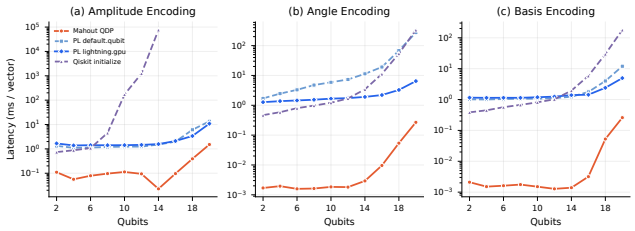



**Figure 3: Data-to-state latency scaling (ms/vector, log scale) for amplitude, angle, and basis encoding across four frameworks. QDP (orange) achieves orders-of-magnitude lower latency than both CPU and GPU backends of existing frameworks.**

even against PennyLane's GPU-accelerated `lightning.gpu` backend. For angle encoding at 20 qubits, QDP (0.27 ms) is 1,018× faster than PennyLane `default.qubit` (274 ms) and 24× faster than `lightning.gpu` (6.5 ms). For basis encoding, the gap is similarly large: QDP (0.26 ms) vs Qiskit (180 ms) at 20 qubits. Qiskit's circuit-based amplitude encoding (`initialize()`) grows exponentially, reaching 171 ms at just 10 qubits, while QDP remains at 0.11 ms. The advantage stems from QDP's fused kernel approach: where PennyLane and Qiskit apply $O(n)$ to $O(n^2)$ sequential gate operations, QDP computes all $2^n$ amplitudes in a single parallel kernel launch.

**Correctness.** Since QDP bypasses circuit simulation with fused CUDA kernels, correctness validation is essential. We compare QDP's encoded states against Qiskit across all three encoding methods (amplitude, angle, basis) and 10 qubit counts from 2 to 20, using float64 precision with 50 random input vectors per configuration. Table 1 shows the maximum amplitude difference is below $10^{-16}$ (machine epsilon) in every case, confirming that QDP produces numerically identical quantum states despite using a fundamentally different computation path.

**Table 1: Max amplitude difference: QDP vs Qiskit (float64).**

| Qubits | Amplitude | Angle | Basis |
|---|---|---|---|
| 4 | $1.1 \times 10^{-16}$ | $2.2 \times 10^{-16}$ | 0 |
| 8 | $5.6 \times 10^{-17}$ | $2.8 \times 10^{-16}$ | 0 |
| 12 | $1.4 \times 10^{-17}$ | $8.3 \times 10^{-17}$ | 0 |
| 16 | $6.9 \times 10^{-18}$ | $4.2 \times 10^{-17}$ | 0 |
| 20 | $1.7 \times 10^{-18}$ | $1.7 \times 10^{-17}$ | 0 |

**Case study: SVHN classification.** To validate end-to-end impact, we train an IQP-based variational quantum classifier on the SVHN dataset [7] (binary classification: digits 1 vs 7, 200 samples, 200 training iterations) across 4–10 qubits. We compare four configurations: QDP and PennyLane, each with CPU and GPU training backends. QDP performs one-shot encoding (~10 ms total for all samples regardless of qubit count), while PennyLane re-encodes via IQP gates on every forward pass. All four configurations converge to identical test accuracy (0.675), confirming that QDP's fused

kernel is functionally equivalent to PennyLane's gate-by-gate simulation in downstream QML tasks. Table 2 shows QDP-GPU achieves up to 1.51× training speedup over PennyLane `lightning.gpu`. The speedup grows with qubit count because IQP circuits have $O(n^2)$ parametric gates; at 10 qubits, QDP eliminates 55 gate operations per forward pass.

**Table 2: End-to-end training time (seconds) on SVHN IQP classification. All configs achieve identical accuracy (0.675).**

| Qubits | PL-GPU | QDP-GPU | Speedup |
|---|---|---|---|
| 4 | 44.8 | 36.4 | 1.23× |
| 6 | 65.2 | 43.2 | 1.51× |
| 8 | 80.7 | 59.4 | 1.36× |
| 10 | 100.1 | 71.1 | 1.41× |

## 5 RELATED WORK

**Quantum data encoding.** Schuld and Petruccione [11] provide a comprehensive survey of encoding strategies for QML. Amplitude encoding is the most commonly used method for variational classifiers, as it can represent $2^n$ features with $n$ qubits, but requires $O(2^n)$ operations for state preparation. Angle and basis encodings trade expressiveness for simpler circuit constructions [4]. Preskill [9] discusses the challenges of the NISQ era, where limited qubit counts and noise make efficient classical preprocessing even more critical for practical QML workloads.

**Quantum software frameworks.** PennyLane [3] provides automatic differentiation for hybrid quantum-classical programs and supports multiple backend simulators. Qiskit [10] offers a comprehensive toolchain for circuit construction, transpilation, and execution on IBM hardware. Both treat data encoding as a preprocessing step without GPU acceleration, constructing states one vector at a time on the CPU. Neither framework batches encoding or leverages GPU parallelism for state preparation. QuClassi [12] demonstrated hybrid quantum-classical architectures at MLSys but did not address the data loading bottleneck. Q2O [6] applied quantum optimization to database query planning at VLDB 2025, demonstrating growing interest in quantum-data systems in the database community.

**Classical data loading systems.** The data starvation problem is well-studied in classical ML: PyTorch DataLoader [8] introduced multi-process prefetching and pinned memory transfers, NVIDIA DALI provides GPU-accelerated image decoding, and tf.data pipelines overlap I/O with computation. QDP brings analogous design principles to the quantum data encoding domain, including asynchronous prefetching, pinned buffer pooling, dual-stream pipelining, and zero-copy tensor output. While these techniques are mature in the classical ML ecosystem, to our knowledge QDP is the first system to apply them to quantum state encoding.

## 6 CONCLUSION

We demonstrated QDP, a GPU-accelerated quantum data processing engine that addresses the data loading bottleneck in QML pipelines. By implementing the full path from columnar storage to GPU-resident quantum states in Rust and CUDA, QDP achieves up to 1,000× lower latency than PennyLane and Qiskit, and 24× speedup over PennyLane's GPU-accelerated `lightning.gpu` backend at 20 qubits. Key engineering contributions include dual-stream pipelining for overlapping host-to-device transfers with GPU encoding, a pinned buffer pool with RAII-based lifecycle management, and zero-copy DLPack interop with PyTorch.

QDP is open source as part of Apache Mahout and supports cloud-native workflows through streaming Parquet ingestion from S3 and GCS. Future directions include: (1) multi-GPU encoding with NCCL-based distribution; (2) AMD ROCm/HIP backend for vendor-neutral acceleration; (3) broader cloud data lake support (Azure Blob, MinIO, Delta Lake/Iceberg); (4) parameterized circuit encoding; and (5) integration with quantum hardware backends for NISQ devices.

## ACKNOWLEDGMENTS

The authors initiated the QDP project within Apache Mahout and serve as PMC members and committers. We thank Chia-Ping Tsai, Andrew Musselman, and Trevor Grant for their mentorship and guidance on the Apache Mahout project. We also thank the Mahout community and the Apache Taipei[1] community for their support.

## REFERENCES

[1] Apache Arrow contributors. 2024. Apache Arrow: A Cross-Language Development Platform for In-Memory Analytics. https://arrow.apache.org.
[2] Apache Parquet contributors. 2024. Apache Parquet: An Open Columnar Storage Format. https://parquet.apache.org.
[3] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, et al. 2022. PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations. In *arXiv:1811.04968*.
[4] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. 2017. Quantum Machine Learning. *Nature* 549 (2017), 195–202.
[5] DLPack contributors. 2023. DLPack: Open In Memory Tensor Structure. https://github.com/dmlc/dlpack.
[6] Hanwen Liu, Federico Spedalieri, and Ibrahim Sabek. 2025. A Demonstration of Q2O: Quantum-augmented Query Optimizer. In *Proceedings of the VLDB Endowment*, Vol. 18.
[7] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. 2011. Reading Digits in Natural Images with Unsupervised Feature Learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*.
[8] Adam Paszke, Sam Gross, Francisco Massa, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*.
[9] John Preskill. 2018. Quantum Computing in the NISQ Era and Beyond. *Quantum* 2 (2018), 79.
[10] Qiskit contributors. 2024. Qiskit: An Open-source Framework for Quantum Computing. (2024). https://doi.org/10.5281/zenodo.2573505
[11] Maria Schuld and Francesco Petruccione. 2021. Machine Learning with Quantum Computers. *Quantum Science and Technology* (2021).
[12] Samuel A. Stein, Betis Baheri, Daniel Chen, Ying Mao, Qiang Guan, Ang Li, Shuai Xu, and Caiwen Ding. 2022. QuClassi: A Hybrid Deep Neural Network Architecture based on Quantum State Fidelity. In *Proceedings of Machine Learning and Systems (MLSys)*.

[1] https://github.com/opensource4you